Operating System Security

Secure Virtual Machine Systems

Secure Virtual Machine Systems

Most of this material is from Chapter 11 of “Operating System Security”, by T. Jaegar, Morgan Claypool Publishers, 2008.

A virtual machine system enables multiple OS instances and their applications to run concurrently on a single physical machine.

Each OS instance runs in a virtualized environment that emulates a physical platform, called a virtual machine (VM).

- Each OS manages the state of its applications and e.g., files, processes, etc., but not the physical hardware.

Virtual Machine Monitors (VMMs)

A VMM multiplexes physical system resources among the operating systems in the VMs.

The VM's OS no longer controls the use of system hardware, but instead receive hardware access only via an indirection through the VMM.

Only the VMM has access to system hardware directly.

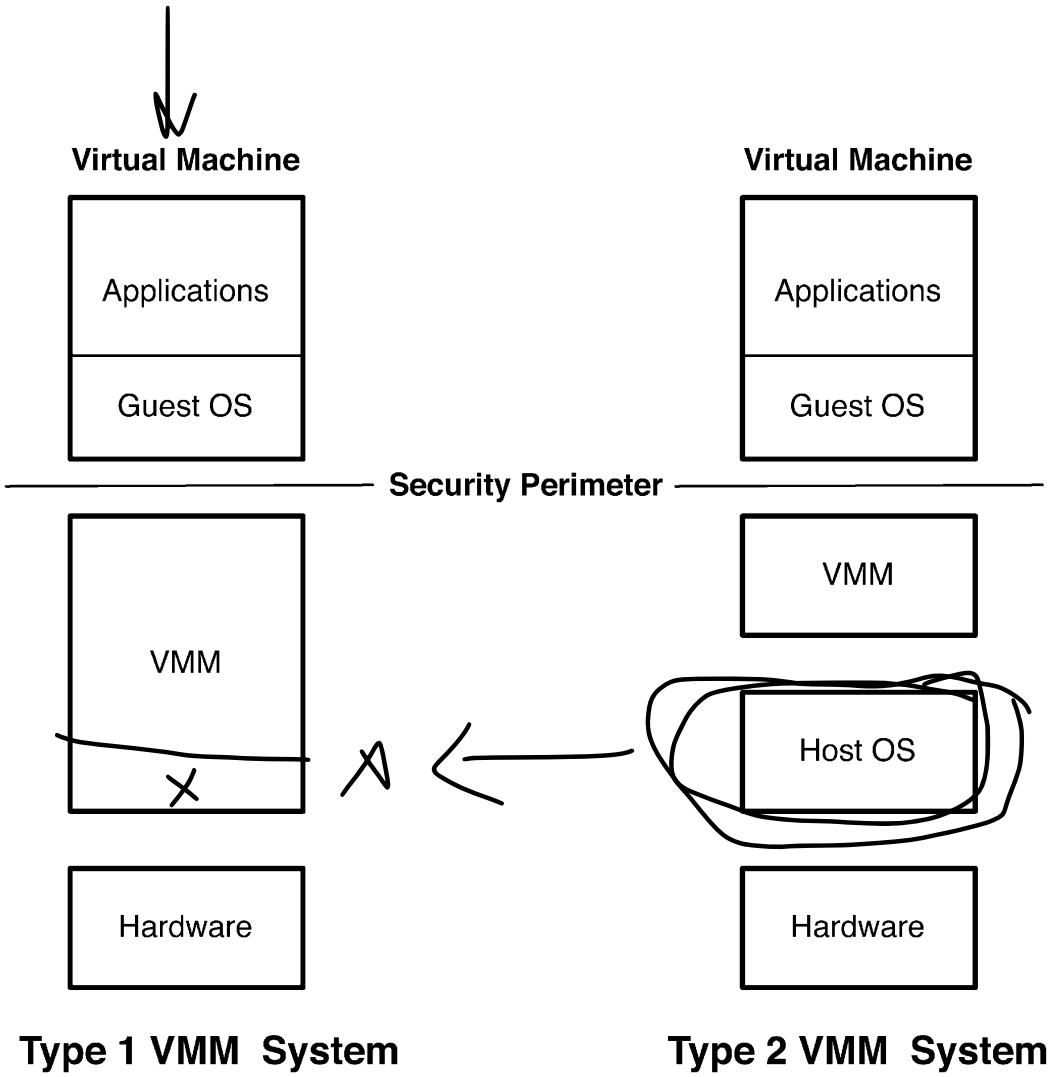
Sometimes VMMs are also called hypervisors

VMM Architectures

Type 1 runs directly on the hardware (e.g., Xen and KVM)

Type 2 requires the services of a host OS in order to run (e.g., VMWare Workstation and VirtualBox).

Which of these VMM types has a smaller TCB?



Security Advantages of VM systems

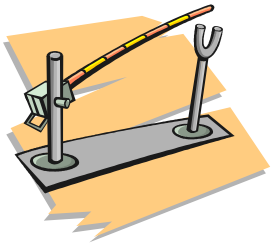
VM systems offer the potential of reducing the size of the TCB, for some Type 1 VMMs

- The VMM is the TCB.
- Note that some Type 1 VMMs do not necessarily offer a smaller TCB code base.

VM systems provide an additional, coarser-grained layer of control for securing a system.

- The hope is that this will allow simpler mediation and simpler policies

Simpler Mediation?



VMM reference monitors only need to mediate the distribution of resources among VMs and inter-VM communication.

Since resources are typically partitioned (not shared) among VMs, securing resource distribution is often simpler

Once a VM is started, only communication with other VMs needs to be mediated

Typically, only a small number of primitives enable inter-VM communication.

Simpler Policies?

The VMM policy will likely be simpler than a secure OS policy.

- The number of VMs is smaller than the number of processes running on an OS
- The number of possible inter-VM communications is smaller than the number of resources on a traditional system

Challenges Building Secure VM Systems

VM systems may generate problems for administrators as there will be many more VMs to administer than physical machines.

The ability to save, restore, and migrate VMs may generate difficulties in ensuring that the VM software base is current and consistent with organizational requirements

Data leaks of VMs into various physical systems and the integrity impact of individual systems into VMs may be difficult to track.

VAX VMM Security Kernel

A VM system that runs untrusted VMs in such a manner that it can control all inter-VM communications, even covert channels.

It is a Type 1 VMM, running directly on hardware.

The VMM includes services for storage (on tape and disk) and for printing.

Each VM can run at its own clearance, and the VMM includes a reference monitor that ensures any use of physical resources is mediated according to a MAC policy.

VAX VMM Security Kernel (con't)

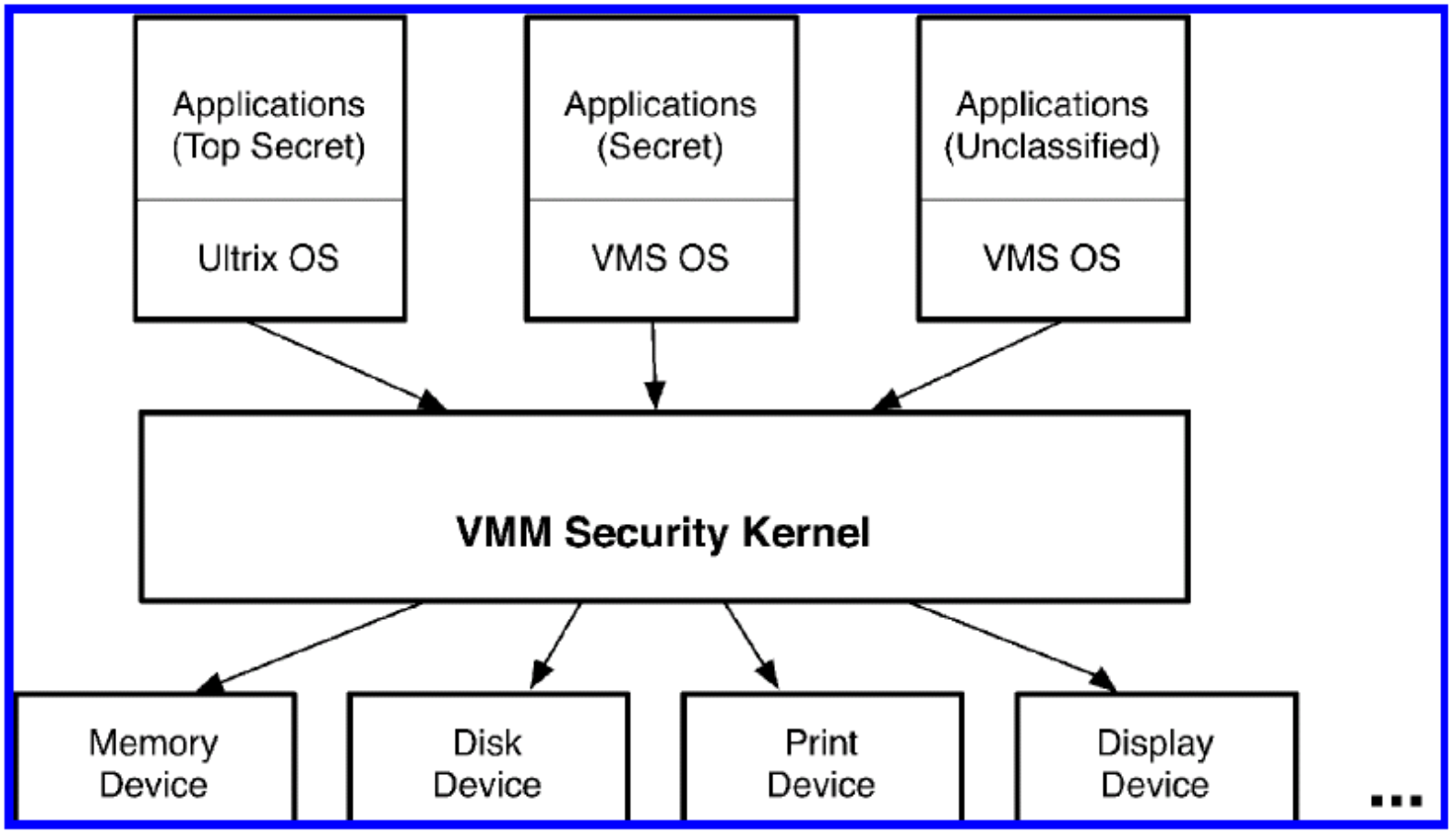


Figure 11.3: The VAX VMM System Architecture: *virtual machines* are labeled according to the secrecy of the information that they can obtain through the *VMM security kernel* to the system's physical devices.

VAX VMM Security Kernel (con't)

The VMM provides an abstraction called "virtual disks" which partition the physical disk into isolated chunks

- This allows two VMs at different clearances to share a single physical disk.

The VMM enforces both secrecy and integrity requirements upon its VMs – versions of BLP and Biba are supported.

The VMM is designed in layers, such that no layer depends on functionality provided by a higher (less-trusted) layer.

VAX VMM Result

The project was cancelled in March 1990.

No reason was given…

Apparently it was discarded just as it became ready for commercial deployment.

- Too bad they didn't share their experience with others aiming to build high-security software.

IBM's Processor Resource/ Systems Manager (PR/SM)

Pronounced "Prism"

A type 1 hypervisor that is capable of running a variety of OS's.

Enables a security administrator to configure VMs such that complete isolation is ensured

In 2005, it was evaluated to EAL 5 (an assurance level – more on this later).

It does not aim for MLS as it's focus is isolation.

VMWare

Security is enabled by its dynamic translation of resource requests (memory, CPU, etc.) into virtualized commands that VMWare VM monitors can mediate.

Resources are grouped into resource pools which partition the CPU and memory resources .

- This enables isolation, but if one VM can delegate control of a pool resource to another VM, then a compromise may lead to an information flow violation.

Supports the introspection of guest VMs from the protected VMM.

NetTop

Built on a Type 2 VMWare system that uses SELinux as the host OS.

Individual VMs can be isolated on the same physical machine

- a single physical machine can be used to connect to multiple isolated networks.

SELinux is used to ensure that any inter-VM communication is authorized according to the MAC policy.

- E.g., a VM is only authorized to access files with a corresponding label

Xen

Provides similar isolation guarantees as VMWare and mediates communication between VMs.

It provides MAC at the VMM level

- The reference monitor implementation in Xen is an ongoing project, but the aim is similar to the LSM interface in Linux.

Uses a privileged VM to provide I/O emulation to the untrusted VMs.

- This VM partitions disk and memory resources and multiplexes network resources among its VMs.
- This puts a lot of trust in this VM (which runs Linux).

Kernel-based Virtual Machine (KVM)

Lets you turn Linux into a Type 1 hypervisor

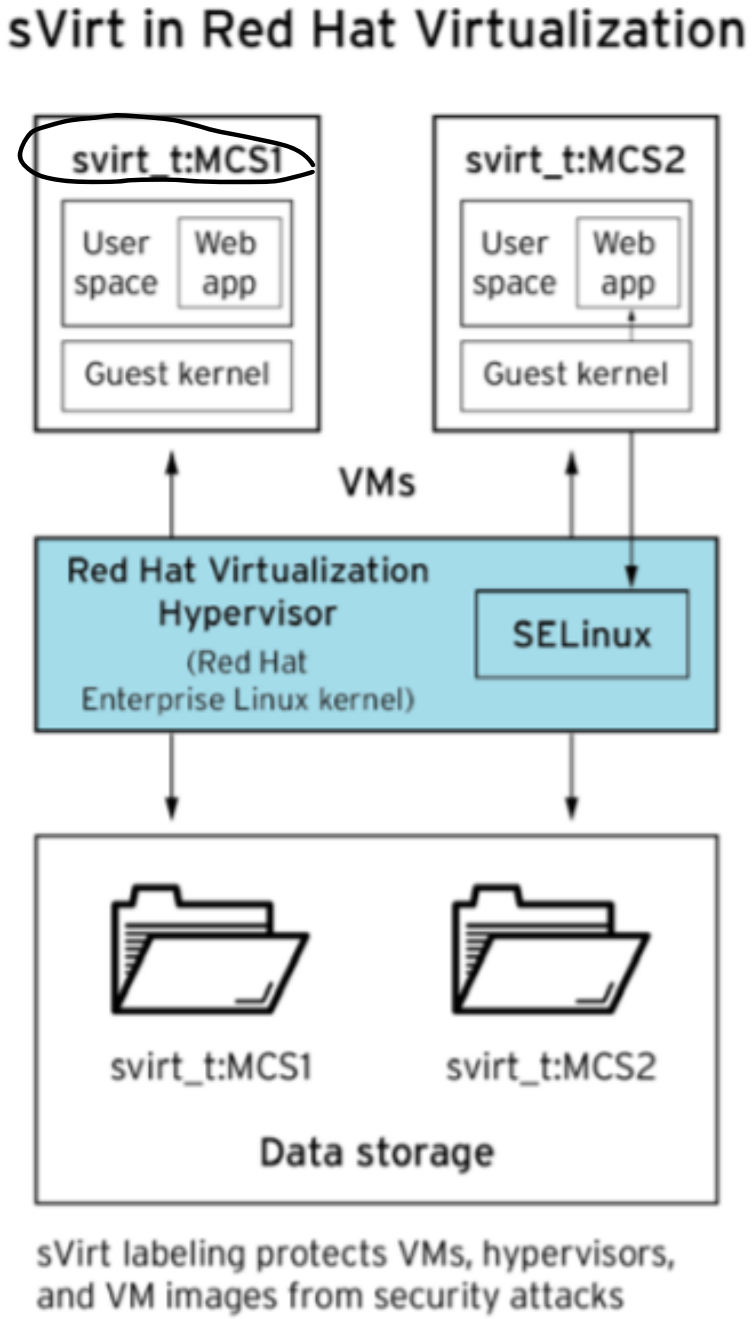
It's part of Linux.

- Uses its memory manager, process scheduler, input/output (I/O) stack, device drivers, security manager, a network stack, and more—to run VMs.
- Every VM is implemented as a regular Linux process, scheduled by the standard Linux scheduler

Virtualizes e.g., the network card, graphics adapter, CPU(s), memory, and disks.

KVM with sVirt

- sVirt uses SELinux type label *svirt_t* plus MCS for isolating VMs from each other.
- Note that according to some other (more recent) documentation [**] the content's type is *svirt_image_t*. This makes more sense! Why?



sVirt labeling protects VMs, hypervisors, and VM images from security attacks

[*] Image from "Operate Red Hat Virtualization and KVM securely with sVirt", https://www.redhat.com/en/resources/secure-virtualization-with-svirt, last accessed March 12, 2020

[**] "sVirt Labeling", https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/7/html/virtualization_security_guide/sect-virtualization_security_guide-svirt-labels, last accessed March 24, 2025

Example output from Linux system

In this example, the:

- Sensitivity level is s0 (default, since it typically only uses one single sensitivity level).
- MCS level (i.e., the category) is c87,c520

```
# ps -eZ | grep qemu-kvm

system_u:system_r:svirt_t:s0:c87,c520 27950 ?  00:00:17 qemu-kvm
```

```
# ls -lZ /var/lib/libvirt/images/*

  system_u:object_r:svirt_image_t:s0:c87,c520   image1
```

What can sVirt do?

FYI, if you feel like experimenting (much like in our previous assignments), see:

Dan Walsh, “**Fun with sVirt.”,** https://danwalsh.livejournal.com/44090.html, last accessed March 24, 2025.

Reflection Questions

What are some security advantages of using virtual machine systems?

Do you believe the general idea that using virtual machine systems will provide better security?

- Why or why not?
- For which systems?

What about if you have a single service that depends on other services running on your system. You can't have them isolated because of dependencies.

- Is there a benefit from running in a VM?

Containers

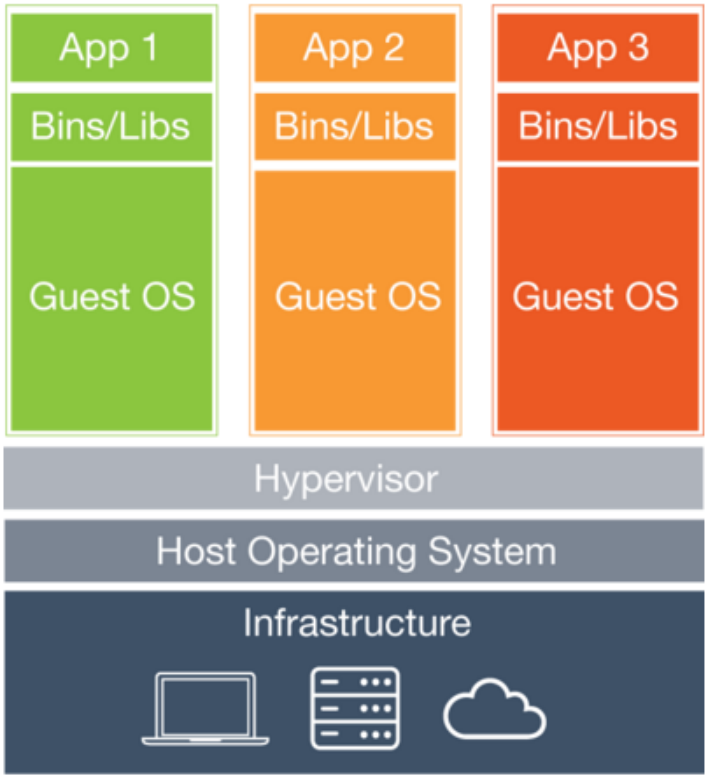
E.g., Docker: https://www.docker.com (last accessed March 24, 2025).

They wrap up a piece of software in a complete filesystem that contains everything it needs to run

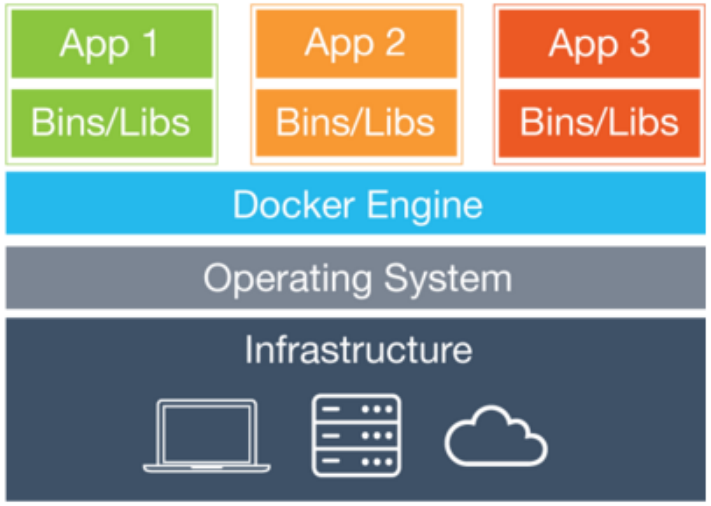
- E.g., code, runtime, system tools, system libraries – anything you can install on a server.
- This guarantees that it will always run the same, regardless of the environment it is running in.

Makes code more portable, lessens burden of configuring software and worrying about conflicting dependencies between software on the same system.

Containers vs VMs



Virtual Machines



Containers

- Image source: http://www.docker.com, accessed April 4, 2016 (intentionally used vs. current image).

- Similar resource isolation/allocation benefits
- Containers are more portable and efficient.

Docker with SELinux - Project Atomic

- Primary reference:
  - http://www.projectatomic.io/docs/docker-and-selinux/ , last accessed March 24, 2025.
  - Note that Project Atomic has since been integrated into Fedora CoreOS: https://docs.fedoraproject.org/en-US/fedora-coreos/faq/ , last accessed March 24, 2025.

- Policy is focused on two concerns: protection of the host, and protection of containers from one another.

- Protection of host is done with TE.

- Protection of containers from one another is done with multi-lateral security, aka multi-category security (MCS).

TE for Docker

Default type for a confined container process:
`svirt_lxc_net_t`

- This type is permitted to read and execute all files types under /usr and most types under /etc.
- It is permitted to use the network but is not permitted to read content under /var, /home, /root, /mnt …
- It is permitted to write only to files labeled `svirt_sandbox_file_t` and `docker_var_lib_t`.

All files in a container are labeled by default as `svirt_sandbox_file_t`.

MCS for Docker

A unique value is assigned to the category field of the SELinux label of each container.

The MCS Labels of the process must dominate the MCS label of the target (usually a file).

Summary

VM systems offer the opportunity for simpler mediation and simpler policies

These simpler policies typically aim to achieve isolation between applications on the system

In both VMs and Container systems, SELinux can be used to have stronger isolation guarantees.

- MCS policies allow for this being invisible to developers & administrators!
- Does this mean that no one needs to write SELinux TE policies anymore?